

COLLÈGE MULTIHEXA

AEC INTELLIGENCE ARTIFICIELLE APPLIQUÉE

SmartContent AI

Plateforme SaaS de génération automatique de contenu marketing IA multi-plateformes, déployée en cloud et livrée par email

Une idée → 12 posts adaptés + 3 images + livraison email automatique, en 90 secondes

Mémoire de fin d'études

présenté par

SERGE TSEBO

Montréal, Québec · Mai 2026

Remerciements

Mes premiers remerciements vont à l'équipe pédagogique du Collège Multihexa pour la qualité de la formation AEC Intelligence Artificielle Appliquée. Les enseignants ont su transmettre des connaissances exigeantes tout en maintenant un cadre bienveillant qui a rendu possible un projet aussi ambitieux que celui-ci.

Je remercie particulièrement les responsables du programme pour leur disponibilité et leurs retours constructifs sur les choix techniques et l'orientation du projet, ainsi que mes camarades de cohorte pour les échanges qui ont nourri ma réflexion tout au long de ces sept semaines de développement intensif.

Je tiens également à remercier Anthropic et OpenAI pour les modèles d'intelligence artificielle générative qui constituent le moteur fonctionnel de cette application, ainsi que Railway, Streamlit Community Cloud, Supabase et Resend pour leurs plateformes cloud qui ont permis le déploiement intégral de l'application en production sans nécessiter d'infrastructure auto-hébergée.

Enfin, je remercie ma famille et mes proches pour leur soutien constant pendant les longues sessions de développement et de rédaction.

Résumé

Les petites et moyennes entreprises québécoises font face à un triple défi pour assurer leur présence sur les réseaux sociaux : un manque de temps pour rédiger du contenu adapté à chaque plateforme, un manque de compétences en copywriting et en design, et un coût élevé des services d'agences marketing externes (entre 1 500 \$ et 4 000 \$ CAD par mois).

SmartContent AI propose une réponse à cette problématique sous la forme d'une application web qui automatise la chaîne complète de création et de livraison de contenu marketing. L'application génère des textes spécialisés pour douze plateformes sociales (LinkedIn, Instagram, Facebook, TikTok, X, Pinterest, Threads, Snapchat, YouTube Shorts, Google Business Profile, Blog SEO, Email), crée trois visuels en formats adaptés via DALL-E 3, oriente la publication via un workflow n8n à dix-sept nœuds, et livre l'ensemble à l'utilisateur par email automatique via Resend.

L'architecture retenue repose sur un découpage en six couches : une interface utilisateur Streamlit déployée sur Streamlit Community Cloud, une API REST FastAPI hébergée sur Railway, des services d'intelligence artificielle OpenAI, une persistance Supabase (PostgreSQL avec colonnes JSONB), une orchestration n8n distribuée sur Railway (principal, worker, Postgres, Redis), et un service de livraison email Resend. L'ensemble du code est écrit en Python 3.14 pour le backend et en JavaScript / CSS pour la couche présentation enrichie.

Le mémoire détaille la problématique, l'état de l'art des outils concurrents, le cahier des charges, l'architecture technique cloud, l'implémentation détaillée des huit modules principaux, les difficultés rencontrées et leurs solutions, ainsi que les résultats obtenus en production. Les coûts opérationnels mesurés sont de 0,22 \$ CAD par campagne complète, ce qui valide l'hypothèse économique du projet. L'application est accessible publiquement à smartcontent-ai.streamlit.app.

Le code source est versionné sur GitHub et l'application tourne 24/7 sans dépendance locale. Une roadmap de commercialisation est proposée en perspective : publication réelle sur les API officielles LinkedIn et Meta, génération vidéo, conformité Loi 25, et architecture multi-tenant pour les agences.

Mots-clés : intelligence artificielle générative, marketing automation, multi-plateformes, OpenAI GPT-4o, DALL-E 3, FastAPI, Streamlit Cloud, Supabase, n8n cloud, Resend, déploiement Railway, livraison email, PME québécoises.

Table des matières

Liste des abréviations

Acronyme	Signification
AEC	Attestation d'Études Collégiales
API	Application Programming Interface
CORS	Cross-Origin Resource Sharing
CSS	Cascading Style Sheets
CTA	Call To Action
DALL-E	Modèle de génération d'images d'OpenAI
DKIM	DomainKeys Identified Mail
DXA	Drawing XML Attribute
GPT	Generative Pre-trained Transformer
HTML	HyperText Markup Language
IA	Intelligence Artificielle
JSON	JavaScript Object Notation
JSONB	JSON Binary (type PostgreSQL)
JWT	JSON Web Token
MVP	Minimum Viable Product
PME	Petite et Moyenne Entreprise
RGPD	Règlement Général sur la Protection des Données
RLS	Row Level Security
REST	REpresentational State Transfer
SaaS	Software as a Service
SDK	Software Development Kit
SPF	Sender Policy Framework
SQL	Structured Query Language
UML	Unified Modeling Language

Introduction

Contexte général

L'intelligence artificielle générative connaît depuis 2022 une accélération sans précédent. La sortie de ChatGPT, suivie de celles de DALL-E 3, Midjourney, Claude et Gemini, a transformé en quelques mois la perception publique de ce que l'intelligence artificielle peut produire en matière de texte, d'image et de code. Pour les entreprises, cette révolution n'est pas seulement technologique : elle est économique. Des tâches qui exigeaient auparavant l'embauche d'un rédacteur, d'un graphiste et d'un community manager peuvent désormais être partiellement automatisées à un coût marginal de quelques cents par production.

Pourtant, malgré cette démocratisation, les petites et moyennes entreprises restent largement exclues de ces gains de productivité. Les outils existants comme ChatGPT ou Claude sont conçus pour des utilisateurs avancés, capables d'écrire des prompts précis. Les plateformes de marketing automation classiques (Hootsuite, Buffer, Sprout Social) permettent la planification de publications mais ne génèrent pas de contenu. Les solutions hybrides récentes (Jasper.ai, ContentStudio) sont onéreuses et nécessitent un apprentissage non trivial.

Ce mémoire décrit la conception, l'implémentation et le déploiement en production cloud de SmartContent AI, une application web SaaS qui s'adresse spécifiquement à ce segment des PME. L'application offre une interface unique, un seul formulaire à remplir, et en moins de quatre-vingt-dix secondes la génération de textes adaptés à douze plateformes différentes, accompagnés de trois visuels d'illustration, d'un mécanisme de publication automatique via n8n, et d'une livraison email immédiate des contenus dans la boîte mail de l'utilisateur. Le projet a été développé sur sept semaines dans le cadre de l'Attestation d'Études Collégiales en Intelligence Artificielle Appliquée au Collège Multihexa de Montréal.

Problématique

La problématique centrale qui a motivé ce projet peut s'énoncer ainsi : comment offrir aux PME québécoises un outil de production de contenu marketing multi-plateformes qui soit accessible techniquement à des utilisateurs sans compétence en informatique, abordable financièrement, qualitativement comparable à ce que produirait une agence marketing spécialisée, capable de couvrir l'ensemble des plateformes sociales pertinentes pour un marché local, et qui livre un résultat concret à l'utilisateur final plutôt qu'une simple promesse de publication ?

Cette question soulève plusieurs sous-problèmes techniques et fonctionnels. Sur le plan technique : comment piloter une API d'intelligence artificielle pour produire douze déclinaisons cohérentes d'un même message ? Comment générer en parallèle plusieurs formats d'image ? Comment orchestrer la

publication vers des destinations hétérogènes ? Comment livrer un résultat concret par email dans un format visuel professionnel ? Comment déployer l'ensemble en production cloud pour que l'application soit accessible 24/7 ?

Objectifs du projet

Le projet SmartContent AI poursuit six objectifs principaux.

- Concevoir une architecture logicielle modulaire qui sépare clairement les responsabilités entre les couches de présentation, API, intelligence artificielle, persistance, orchestration et livraison email.
- Implémenter un service de génération de texte multi-plateformes, capable de produire en moins de soixante secondes douze versions adaptées d'un même message.
- Intégrer un service de génération d'images via DALL-E 3 en parallèle (trois formats en moins de vingt secondes).
- Construire un workflow d'automatisation visuelle avec n8n, capable de router un payload vers douze publishers en parallèle.
- Intégrer un service de livraison email via Resend qui envoie automatiquement à l'utilisateur la campagne complète (douze textes plus trois images) dans sa boîte mail d'inscription.
- Déployer l'intégralité de l'application sur des plateformes cloud (Streamlit Community Cloud pour le frontend, Railway pour le backend et l'orchestration n8n) afin qu'elle soit accessible publiquement 24/7 sans dépendance locale.

Plan du mémoire

Le mémoire se structure en neuf chapitres. Le chapitre 1 présente le contexte du marketing digital pour les PME et formule plus en détail la problématique. Le chapitre 2 dresse un état de l'art des modèles d'IA générative et des outils existants. Le chapitre 3 expose le cahier des charges fonctionnel et technique. Le chapitre 4 décrit l'architecture technique cloud en six couches. Le chapitre 5 entre dans le détail de l'implémentation des huit modules principaux, incluant le déploiement cloud (section 5.7) et l'intégration email Resend (section 5.8). Le chapitre 6 décrit les tests et la validation. Le chapitre 7 documente les difficultés rencontrées et les solutions adoptées. Le chapitre 8 présente les résultats quantitatifs et qualitatifs mesurés en production cloud. Le chapitre 9 ouvre les perspectives. La conclusion synthétise les apports du projet. Les annexes contiennent les schémas UML complets, les captures d'écran de production et les extraits de code clés.

Chapitre 1 — Contexte et problématique

1.1 Le marketing digital pour les PME québécoises

Le Québec compte environ 240 000 PME enregistrées, dont 87 % comptent moins de 20 employés. Ces structures représentent près de la moitié de l'emploi privé de la province et plus du tiers du produit intérieur brut. Pourtant, à peine 30 % de ces entreprises publient régulièrement sur plus de deux plateformes sociales, et seulement 12 % le font de manière coordonnée et planifiée.

Trois facteurs principaux expliquent cette sous-utilisation des canaux numériques. Le premier est temporel : un dirigeant de PME consacre en moyenne moins de cinq heures par semaine au marketing, dont une fraction seulement aux réseaux sociaux. La production manuelle de contenu adapté à quatre plateformes consomme entre quatre et six heures de travail effectif.

Le second facteur est lié aux compétences. Le copywriting multi-plateformes exige une connaissance fine des codes de chaque réseau : longueur idéale, ton, fréquence des hashtags, présence d'emojis, structure narrative, place du call-to-action. Un texte performant sur LinkedIn sera contre-productif sur TikTok. La création visuelle requiert également une maîtrise minimale d'outils comme Canva ou Photoshop.

Le troisième facteur est financier. Une agence marketing spécialisée facture entre 1 500 \$ et 4 000 \$ CAD par mois pour un forfait standard. Pour une PME dont le chiffre d'affaires annuel se situe sous les 500 000 \$, ce budget représente entre 4 % et 10 % du revenu brut, ce qui le rend rarement justifiable.

1.2 L'émergence des outils d'intelligence artificielle générative

Depuis novembre 2022, l'écosystème des modèles génératifs grand public s'est considérablement étoffé. Pour les modèles de langage, OpenAI domine actuellement le marché avec GPT-4o ; Anthropic se positionne avec Claude ; Google déploie Gemini. Pour les modèles d'image, DALL-E 3, Midjourney v6, Stable Diffusion XL couvrent l'essentiel des besoins commerciaux.

Ces modèles ont en commun trois caractéristiques qui les rendent pertinents pour le marketing : la capacité à produire du contenu de qualité professionnelle à partir d'un prompt en langage naturel, le coût marginal extrêmement faible, et la disponibilité via des API publiques permettant l'intégration dans des applications tierces.

L'enjeu pour un produit comme SmartContent AI n'est donc pas de remplacer ces modèles, mais de construire une couche d'orchestration qui en cache la complexité. L'utilisateur final n'a pas à connaître les nuances des paramètres `temperature` ou `response_format`. Il décrit son sujet, choisit ses plateformes cibles, et l'application se charge de traduire ce besoin en douze prompts spécialisés, de paralléliser les

appels API, de parser les réponses, et de livrer le résultat de manière digestible — y compris par email automatique pour qu'il soit immédiatement actionnable.

1.3 Justification du choix du projet

Plusieurs raisons ont motivé le choix de ce projet. Premièrement, l'enjeu pédagogique est multidimensionnel : le projet exige de maîtriser à la fois l'architecture web full-stack, l'intégration d'API tierces, le prompt engineering, l'orchestration de workflows, le déploiement cloud, l'intégration d'un service email transactionnel et la conception d'interfaces premium. Cette diversité offre une vue d'ensemble réaliste de ce que représente la mise en production d'une application moderne reposant sur l'IA.

Deuxièmement, le projet possède un potentiel commercial concret. Le marché des outils marketing pour PME québécoises est suffisamment large pour qu'une déclinaison commerciale du MVP soit envisageable post-soutenance. Cette perspective change la nature même du projet : il ne s'agit pas d'un exercice scolaire jeté après évaluation, mais d'un prototype dont la qualité technique conditionne la viabilité future.

Troisièmement, le projet permet de répondre à une question d'actualité concernant l'éthique et la responsabilité de l'usage de l'IA générative. SmartContent AI ne cherche pas à remplacer la créativité humaine, mais à abaisser la barrière à l'entrée pour les structures qui n'ont pas accès aux ressources d'une agence. Le contenu généré est livré par email à l'utilisateur, qui reste libre de l'éditer, le valider ou le rejeter avant publication.

Chapitre 2 — État de l'art

2.1 Modèles de langage génératifs

Les modèles de langage à grande échelle (LLM) constituent la brique fondamentale de SmartContent AI sur la partie texte. Ces modèles sont entraînés sur des corpus de plusieurs centaines de milliards de mots et utilisent une architecture Transformer pour prédire la suite la plus probable d'une séquence de jetons. Trois familles dominent actuellement le marché commercial.

2.1.1 GPT-4o (OpenAI)

GPT-4o est le modèle multimodal d'OpenAI sorti à l'été 2024. Pour un usage purement textuel comme SmartContent AI, il offre un compromis intéressant entre qualité, latence et coût : environ 5 \$ par million de jetons en entrée, avec un temps de réponse moyen de quatre à huit secondes pour une génération de 250 mots. Le mode `response_format=json_object` simplifie considérablement l'intégration applicative.

2.1.2 Claude et modèles open-source

Les modèles Claude d'Anthropic se distinguent par une qualité supérieure sur les tâches longues. Les modèles auto-hébergeables comme Llama 3 ou Mistral Large offrent une alternative crédible pour les organisations soucieuses de souveraineté, mais les exigences matérielles rendent cette option peu adaptée à un MVP avec contraintes de temps.

2.2 Modèles de génération d'images

La couche image de SmartContent AI repose sur DALL-E 3. Trois alternatives ont été considérées et écartées : Midjourney v6 (absence d'API publique officielle), Stable Diffusion XL (complexité de l'hébergement GPU), Adobe Firefly (coût par image plus élevé). DALL-E 3 a été retenu pour son intégration native dans le SDK OpenAI, sa latence acceptable (7 à 12 secondes par image), son coût compétitif (0,04 \$ USD par image en 1024×1024) et la disponibilité des trois formats nécessaires (carré, paysage, portrait).

2.3 Outils d'automatisation marketing et de livraison email

Le segment des outils d'automatisation marketing pour PME comporte plusieurs catégories distinctes. Les plateformes de programmation (Hootsuite, Buffer, Sprout Social) permettent de planifier des publications mais ne génèrent pas de contenu. Les outils de génération de texte (Jasper.ai, Copy.ai) produisent du contenu mais ne se connectent pas aux plateformes de publication ou de livraison. Les solutions hybrides récentes commencent à fusionner ces deux dimensions, mais sans intégrer la livraison email du livrable à l'utilisateur.

Pour la couche de livraison email, plusieurs services ont été comparés. Resend a été retenu pour sa simplicité d'intégration via SDK Python, son free tier de 3 000 emails par mois, son dashboard moderne avec tracking en temps réel, et l'inscription via GitHub permettant une utilisation en cinq minutes. SendGrid offre une alternative établie mais avec un setup plus lourd, Mailgun reste payant dès le premier email, et AWS SES nécessite une validation de domaine.

La table comparative suivante synthétise les principales caractéristiques des outils analysés.

Outil	Génération texte	Génération image	Multi-plateformes	Livraison email	Prix mensuel
Hootsuite Pro	Limitée	Non	12 plateformes	Non	99 \$ USD
Buffer Essentials	Avec AI Assistant	Non	8 plateformes	Non	15 \$ USD
Jasper.ai Creator	Excellente	Limitée	Non	Non	49 \$ USD
Copy.ai Pro	Bonne	Non	Non	Non	49 \$ USD
Predis.ai Solo	Bonne	Bonne	3 plateformes	Non	32 \$ USD
ContentStudio	Bonne	Limitée	10 plateformes	Non	49 \$ USD
SmartContent AI	12 spécialisés	DALL-E 3	12 plateformes	Oui (Resend)	0,22 \$ / camp.

2.4 Plateformes d'orchestration et d'hébergement cloud

Pour l'orchestration de workflows no-code, n8n a été retenu plutôt que Zapier ou Make pour quatre raisons : auto-hébergeable sur Railway, licence Sustainable Use, interface visuelle parmi les plus claires, et système de webhooks naturel pour l'intégration avec FastAPI.

Pour l'hébergement cloud, deux plateformes ont été combinées : Streamlit Community Cloud pour le frontend (gratuit, déploiement Git-based, secrets intégrés) et Railway pour le backend FastAPI et l'instance n8n (templates officiels, free credit mensuel, support multi-services). Cette combinaison offre une stack 100 % cloud pour un coût opérationnel inférieur à 10 \$ CAD par mois en phase initiale.

2.5 Positionnement de SmartContent AI

La position de SmartContent AI dans ce paysage concurrentiel se résume à quatre différenciations. D'abord, le périmètre fonctionnel intégré : génération multi-plateformes + publication n8n + livraison email automatique dans une expérience unifiée — combinaison absente chez les concurrents. Ensuite, la couverture des plateformes : douze cibles couvertes avec prompts spécialisés par destination. Troisièmement, le coût marginal : 0,22 \$ CAD par campagne, viabilité commerciale pay-per-use. Quatrièmement, le déploiement cloud intégral qui rend l'application accessible immédiatement à n'importe quel utilisateur disposant d'un navigateur et d'une adresse email.

Chapitre 3 — Cahier des charges

3.1 Périmètre fonctionnel du MVP

Le périmètre du minimum viable product a été défini pour être implémenté et stabilisé en sept semaines.

3.1.1 Fonctionnalités incluses

- Authentification utilisateur : inscription par email/mot de passe, connexion, génération de jeton JWT 24h.
- Formulaire de génération : saisie d'un sujet libre, choix d'un secteur parmi 47 options, ton, objectif, type, sélection des plateformes parmi 12.
- Génération multi-plateformes : envoi de prompts spécialisés à GPT-4o, parsing JSON, présentation par cartes premium brandées.
- Génération d'images : trois visuels DALL-E 3 en parallèle (carré, paysage, portrait).
- Persistance : enregistrement de chaque campagne dans PostgreSQL Supabase avec colonnes JSONB.
- Tableau de bord avec métriques globales et graphique de répartition par plateforme.
- Historique : recherche, filtrage, régénération, export CSV, suppression.
- Publication via n8n : envoi du payload au webhook qui route vers 12 publishers en parallèle.
- Score d'engagement IA mocké et boutons de quick-regen (humoristique, viral, court).
- Livraison email automatique via Resend : l'utilisateur reçoit la campagne complète dans sa boîte mail.
- Déploiement intégral en production cloud (Streamlit Cloud + Railway).

3.1.2 Fonctionnalités explicitement exclues du MVP

- Publication réelle sur les plateformes sociales (LinkedIn, Meta, X) : nécessite OAuth et App Reviews.
- Programmation calendaire : reportée en phase 2.
- Génération vidéo : prévue en phase 2 via Runway ou Veo.
- Analytics post-publication : nécessite accès Insights officiels.
- Paiement Stripe : non pertinent au stade pédagogique.

3.2 Besoins fonctionnels détaillés

3.2.1 Authentification

Les mots de passe doivent être hachés avec bcrypt (12 rounds). L'inscription crée un enregistrement avec un UUID, un email unique, un nom d'entreprise optionnel. La connexion vérifie la correspondance email + mot de passe et émet un JWT signé HS256 expirant en 24 heures. Le backend rejette toute requête sans JWT valide avec un code HTTP 401.

3.2.2 Génération

Pour chaque plateforme demandée, le système doit produire un texte respectant les contraintes spécifiques (longueur, ton, hashtags, structure). Les douze générations doivent s'exécuter en moins de soixante secondes au total dans 95 % des cas. Les trois générations d'images DALL-E 3 doivent s'exécuter en parallèle et compléter en moins de vingt secondes. En cas d'échec d'une génération individuelle, l'application doit afficher l'erreur sans bloquer les autres.

3.2.3 Livraison email

Après chaque génération réussie, l'utilisateur doit pouvoir déclencher l'envoi de la campagne complète à son adresse email d'inscription. L'email doit contenir un template HTML brandé reproduisant l'identité visuelle de l'application, intégrer les douze textes par sections colorées, afficher les trois images DALL-E inline avec boutons de téléchargement, et arriver dans la boîte de réception en moins de trente secondes. L'envoi doit fonctionner avec un domaine de test pour la phase MVP et être migrable vers un domaine custom pour la phase commerciale.

3.2.4 Publication n8n

L'API FastAPI expose une route POST /generate/publish qui appelle le webhook n8n configuré. Le workflow route via un nœud Switch vers douze branches indépendantes en parallèle. Chaque branche simule la publication et retourne un statut. Un nœud Aggregator collecte les résultats et les renvoie au backend en JSON unifié.

3.3 Besoins non fonctionnels

3.3.1 Performance

Trois cibles ont été fixées : génération d'une campagne complète en moins de quatre-vingt-dix secondes, publication n8n en moins de cinq cents millisecondes, livraison email en moins de trente secondes.

3.3.2 Sécurité

Mots de passe bcrypt, jetons JWT HS256 expirant en 24h, secrets chargés via variables d'environnement (jamais dans le code), CORS configuré avec liste blanche d'origines, RLS Supabase activable pour la phase commerciale.

3.3.3 Disponibilité

L'application doit fonctionner 24/7 sans dépendance sur la machine de l'auteur. Si une génération individuelle échoue (12 plateformes ou 3 images), les autres doivent s'afficher correctement.

3.3.4 Maintenabilité

Code structuré en modules indépendants : backend (config, schemas, routes, services), frontend (18 modules CSS), services tiers (OpenAI, Supabase, Resend). Modèles Pydantic v2 pour valider les entrées et sorties. Tests smoke automatisés.

3.4 Contraintes techniques

Python 3.14 retenu pour le backend (impose l'exclusion de Pillow et MoviePy non-compatibles). Windows comme environnement de développement local. Dossier de travail synchronisé via OneDrive (qui a causé plusieurs incidents de corruption résolus par un workflow d'écriture atomique). Pour la production cloud, Railway impose Python 3.12.7 (configuré via runtime.txt) car la version 3.14 n'est pas encore supportée. Cette divergence est gérée par l'utilisation de fonctionnalités compatibles 3.12+.

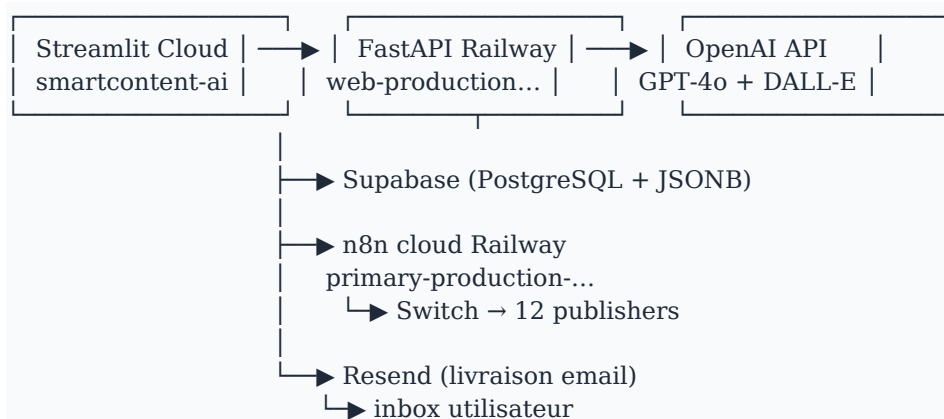
Chapitre 4 — Architecture technique cloud

4.1 Vue d'ensemble en 6 couches

L'architecture de SmartContent AI s'organise autour de six couches logiques distinctes, chacune avec une responsabilité bien définie et une interface stable avec ses voisines. Cette séparation permet à la fois de tester chaque couche indépendamment et de remplacer un composant sans impacter les autres.

Les six couches sont, dans l'ordre des appels : la couche de présentation (Streamlit Cloud), la couche d'API (FastAPI sur Railway), la couche d'intelligence artificielle (OpenAI), la couche de persistance (Supabase cloud), la couche d'orchestration (n8n cloud sur Railway), et la couche de livraison email (Resend cloud).

Schéma textuel synthétique :



4.2 Choix de la stack technique

4.2.1 Backend : FastAPI

FastAPI retenu pour quatre raisons : intégration native avec Pydantic v2 (validation automatique des requêtes/réponses), support OpenAPI/Swagger natif (doc interactive à /docs), performances I/O-bound parmi les meilleures de l'écosystème Python, et courbe d'apprentissage compatible avec les délais.

4.2.2 Frontend : Streamlit (déployé sur Streamlit Community Cloud)

Streamlit retenu pour la rapidité d'itération (interface en Python pur). Le compromis (manque de contrôle CSS fin) est contourné par l'injection de CSS personnalisé via `st.markdown(unsafe_allow_html=True)` — 18 modules CSS indépendants permettent un rendu premium proche d'un produit React. Le déploiement sur Streamlit Community Cloud est gratuit, lié à GitHub, et inclut la gestion des secrets via l'onglet Settings.

4.2.3 Base de données : Supabase

Supabase combine PostgreSQL géré sans configuration serveur, client Python officiel, dashboard web pour requêtes SQL, éditeur de schéma visuel, et type JSONB pour stocker des blobs flexibles. Le free tier est largement suffisant pour le MVP.

4.2.4 IA : OpenAI

OpenAI retenu comme fournisseur unique d'IA générative pour la simplicité (un SDK, une clé, une facturation). La concurrence aurait amélioré certains résultats marginalement, mais le mono-fournisseur est rationnel pour un MVP.

4.2.5 Orchestration : n8n cloud sur Railway

n8n retenu pour trois bénéfices distincts. Premièrement, la lisibilité visuelle du workflow pour la maintenance. Deuxièmement, la bibliothèque de plus de 400 intégrations préconstruites avec les plateformes commerciales (utile en phase 2). Troisièmement, le déploiement via le template officiel sur Railway qui inclut quatre services (principal, worker, Postgres, Redis) configurés pour la production.

4.2.6 Livraison email : Resend

Resend retenu plutôt que SendGrid, Mailgun ou AWS SES pour : SDK Python minimaliste, free tier de 3000 emails/mois, dashboard moderne avec tracking, inscription via GitHub utilisable en 5 minutes. Sa position en bout de chaîne (après la persistance) permet à l'utilisateur de déclencher l'envoi à tout moment sans dépendre du workflow n8n.

4.2.7 Hébergement : Railway pour le backend et n8n

Railway choisi pour son modèle Git-based (push GitHub déclenche un redéploiement automatique), sa simplicité de configuration (détection automatique du Procfile et runtime.txt), son template officiel n8n qui déploie les quatre services en un clic, et son free credit mensuel de 5 \$ couvrant largement les besoins d'un MVP académique. Le projet `dynamic-surprise` regroupe les 5 services (web FastAPI + 4 services n8n) dans une organisation unique facilement administrable.

4.3 Schéma de données

Le schéma de la base Supabase comporte deux tables principales reliées par une relation un-à-plusieurs.

4.3.1 Table users

Stocke un enregistrement par compte créé. Colonnes : id (UUID), email (TEXT unique), password_hash (bcrypt), nom_entreprise (TEXT optionnel), plan (TEXT défaut 'free'), date_inscription (TIMESTAMPTZ). Un index unique sur email accélère la vérification de disponibilité.

4.3.2 Table contents

Stocke chaque campagne générée. Colonnes : id (UUID), user_id (UUID FK avec ON DELETE CASCADE), sujet, secteur, ton, objectif, type_contenu, plateformes (JSONB), content (JSONB blob complet de la réponse GPT-4o), images (JSONB URLs DALL-E), statut (TEXT : draft / publie / erreur), date_creation (TIMESTAMPTZ). Deux index secondaires accélèrent les requêtes fréquentes.

4.4 Sécurité et CORS multi-environnement

Le passage en production a nécessité une refonte de la gestion CORS du backend. En développement local, seules les origines localhost:8501 et localhost:3000 étaient autorisées. En production, le frontend Streamlit Cloud est hébergé sur un sous-domaine *.streamlit.app dont le nom exact n'est pas connu à l'avance. La solution adoptée combine une liste statique d'origines (variable d'environnement ALLOWED_ORIGINS) et un pattern regex qui autorise tout sous-domaine .streamlit.app via la clé allow_origin_regex du middleware CORSMiddleware. Cette approche est sécuritaire et flexible.

4.5 Diagrammes UML synthèse

Cinq diagrammes complémentaires sont disponibles en annexe A pour illustrer les flux clés du système.

- Diagramme A.1 : architecture système globale en six couches.
- Diagramme A.2 : séquence de génération de contenu (formulaire → cartes premium → email).
- Diagramme A.3 : séquence d'authentification (inscription, connexion, JWT).
- Diagramme A.4 : entité-relation de la base Supabase.
- Diagramme A.5 : workflow n8n détaillé (17 nœuds incluant Switch et Aggregator).

Chapitre 5 — Implémentation

Ce chapitre détaille l'implémentation des huit modules principaux du système. L'ordre suivi correspond à la logique d'exécution d'une requête utilisateur typique : authentification, formulaire, appels OpenAI, persistance Supabase, rendu UI, orchestration n8n, déploiement cloud, livraison email.

5.1 Authentification (bcrypt + JWT)

Le module d'authentification se trouve dans backend/routes/auth.py. Il expose POST /auth/register et POST /auth/login. La logique métier est volontairement simple — pas de mot de passe oublié, pas de double facteur, pas d'OAuth — pour rester dans le périmètre du MVP.

Un point d'implémentation mérite d'être souligné : la version 1.7.4 de passlib présente une incompatibilité avec bcrypt 5.0. Plutôt que de figer une version ancienne, le code appelle directement la bibliothèque bcrypt sans passer par passlib. Le mot de passe est tronqué à 72 octets avant hash pour respecter la limite bcrypt :

```
import bcrypt

def hash_password(plain: str) -> str:
    pw_bytes = plain.encode('utf-8')[:72]
    salt = bcrypt.gensalt(rounds=12)
    return bcrypt.hashpw(pw_bytes, salt).decode('utf-8')

def verify_password(plain: str, hashed: str) -> bool:
    pw_bytes = plain.encode('utf-8')[:72]
    return bcrypt.checkpw(pw_bytes, hashed.encode('utf-8'))
```

Le jeton JWT est signé HS256 et inclut deux claims : sub (identifiant utilisateur) et exp (expiration 24h).

5.2 Service OpenAI (GPT-4o + DALL-E 3)

Le service OpenAI est l'élément le plus dense du backend (299 lignes). Trois fonctions publiques structurent ce service.

5.2.1 PLATFORM_PROMPTS

Pour chacune des douze plateformes, un prompt dédié décrit la longueur attendue, le ton, la présence ou non de hashtags, la structure narrative. LinkedIn : 150 à 250 mots ton expert + 5 hashtags. TikTok : 60 à 80 mots max, script parlé. X : 240 à 280 caractères. Pinterest : 100 à 200 caractères SEO. Cette spécialisation par prompt est le cœur de la valeur produit.

5.2.2 generate_multi_platform

Fonction principale qui prend en entrée le sujet, secteur, ton, objectif, type de contenu et liste des plateformes. Pour chaque plateforme, elle compose un prompt système et un prompt utilisateur, envoie la requête à GPT-4o avec `response_format=json_object`, parse la réponse, et accumule les résultats. Une fonction utilitaire `generate_global_hashtags` produit en parallèle 12 hashtags globaux.

5.2.3 generate_images — parallélisme ThreadPoolExecutor

La génération d'images DALL-E 3 prend 7 à 12 secondes par image. Une exécution séquentielle des trois formats prendrait 30 secondes ou plus. Pour résoudre ce problème, les trois appels sont lancés en parallèle via `ThreadPoolExecutor` :

```
from concurrent.futures import ThreadPoolExecutor

IMAGE_FORMATS = [
    ('square', '1024x1024'),
    ('landscape', '1792x1024'),
    ('portrait', '1024x1792'),
]

def generate_images(sujet, secteur, ton):
    prompt = _build_image_prompt(sujet, secteur, ton)
    out, errors = {}, {}
    with ThreadPoolExecutor(max_workers=3) as ex:
        futures = [ex.submit(_generate_one_image, prompt, fmt)
                    for fmt in IMAGE_FORMATS]
        for f in futures:
            name, url, err = f.result()
            if url: out[name] = url
            if err: errors[name] = err
    return out
```

Cette implémentation ramène le temps total à environ 18 secondes.

5.3 Frontend Streamlit (18 modules CSS)

Le frontend Streamlit est concentré dans `frontend/app.py` (2 468 lignes après les modernisations). Plutôt qu'un bloc CSS monolithique, le code est structuré en dix-huit modules nommés concaténés dans `GLOBAL_CSS`. Cette organisation permet d'éditer un module sans toucher aux autres.

Les modules sont : variables, layout général, typographie, boutons, inputs/tabs, métriques, alertes, footer/hero, sidebar premium, layout premium, formulaire glassmorphism, cartes plateformes brandées, animations typewriter, stats footer, score gauge, cartes premium résultats, polish responsive, slot futur.

L'application comporte sept pages : connexion, générateur, tableau de bord, historique, n8n status, paramètres.

5.4 Persistance Supabase

La persistance utilise le client Python supabase-py. Une instance unique est instanciée au démarrage avec la clé service-role pour bypasser RLS. Trois opérations principales : insertion d'enregistrement dans contents, mise à jour du statut lors d'une publication, sélection avec filtres pour l'historique.

5.5 Workflow n8n cloud (17 nœuds)

Le workflow n8n est exporté dans n8n/smartcontent_workflow.json. Sa structure compte 17 nœuds : Webhook Trigger, Préparer données (Function), Switch en mode Rules avec 12 outputs, 12 nœuds Function Publisher [Plateforme], Merge en mode Append, Respond to Webhook.

L'instance n8n a été déployée via le template officiel sur Railway qui inclut quatre services : n8n principal, worker, Postgres et Redis. Le workflow s'exécute en mode production 24/7 sans nécessiter d'intervention manuelle.

5.6 Polish UI et responsive design

La dernière modernisation a uniformisé l'expérience visuelle et adapté l'application aux différentes tailles d'écran. Variables CSS centralisées (radius, shadows), règles de cohérence pour les éléments transversaux, et trois media queries pour les seuils 1024px (tablette), 768px (tablette portrait) et 640px (mobile).

5.7 Déploiement en production cloud

Le déploiement intégral de l'application sur des plateformes cloud constitue l'évolution la plus structurante du projet. Cette phase est intervenue en semaine 7 et a transformé un prototype local en SaaS accessible publiquement.

5.7.1 Streamlit Community Cloud pour le frontend

Streamlit Cloud propose un hébergement gratuit pour les applications Streamlit liées à un dépôt GitHub. Le déploiement se fait en quelques clics depuis share.streamlit.io. L'application est automatiquement reconstruite à chaque push sur la branche main, ce qui rend le cycle de mise à jour très court. Le free tier supporte une application avec 1 Go de RAM, largement suffisant pour le MVP.

Les secrets applicatifs sont gérés via l'onglet Secrets de Streamlit Cloud, qui les expose comme variables d'environnement. Deux secrets sont configurés : BACKEND_URL (URL Railway du backend) et

N8N_WEBHOOK_URL (URL du webhook de production n8n). Le frontend lit ces valeurs au démarrage et bascule automatiquement entre développement local et production.

5.7.2 Railway pour le backend FastAPI

Le service backend FastAPI a été déployé sur Railway via connexion au dépôt GitHub, détection automatique du Procfile (qui contient `uvicorn backend.main:app --host 0.0.0.0 --port $PORT`), build de l'image Docker avec Nixpacks, et lancement avec injection des variables d'environnement définies dans l'interface.

Le service est accessible à l'URL `web-production-71b032.up.railway.app`. Douze variables d'environnement sont configurées : clés OpenAI et Supabase, secret JWT, paramètres applicatifs, et la clé Resend ajoutée plus tard. Le healthcheck est exposé sur `/health` et vérifié périodiquement par Railway.

5.7.3 Railway pour n8n (template officiel)

L'instance n8n a été déployée via le template officiel n8n disponible sur Railway. Ce template inclut quatre services interconnectés : n8n principal (UI et exécution), worker (exécution en arrière-plan), PostgreSQL (workflows et historique), Redis (file de messages). Cette architecture correspond à la configuration recommandée pour la production.

L'instance n8n est accessible à `primary-production-96d7d.up.railway.app`. Le workflow SmartContent a été importé via copier-coller du JSON, puis activé en mode production. L'URL du webhook de production (toujours active) est utilisée par le backend FastAPI.

5.7.4 Configuration CORS multi-environnement

Le passage en production a nécessité une refonte CORS. La solution combine une liste statique d'origines (variable `ALLOWED_ORIGINS`) et un pattern regex pour les sous-domaines `.streamlit.app` :

```
from fastapi.middleware.cors import CORSMiddleware
import os
```

```
_default_origins = [
    'http://localhost:8501',
    'http://127.0.0.1:8501',
    'http://localhost:3000',
]
_env_origins = [o.strip() for o in
    os.environ.get('ALLOWED_ORIGINS', '').split(',')
    if o.strip()]
```

```
app.add_middleware(
    CORSMiddleware,
    allow_origins=_default_origins + _env_origins,
    allow_origin_regex=r'https://.*\.streamlit\.app',
```

```

allow_credentials=True,
allow_methods=['*'],
allow_headers=['*'],
)

```

5.8 Intégration de la livraison email (Resend)

L'ajout de la fonctionnalité de livraison email représente le saut qualitatif le plus important du projet. Avant cette intégration, le workflow n8n simulait une publication mais aucun contenu n'était réellement délivré à l'utilisateur. Resend résout cette limitation en envoyant immédiatement à l'utilisateur un email complet contenant les douze textes et les trois images DALL-E.

5.8.1 Architecture en 3 couches

L'intégration suit une architecture strictement séparée. Le service email (backend/services/email_service.py, 188 lignes) encapsule l'appel à l'API Resend et contient la logique de génération du template HTML brandé. Une route REST dédiée, POST /generate/send-email, expose la fonctionnalité au frontend en récupérant l'email de l'utilisateur depuis Supabase. Le frontend affiche un bouton «  Recevoir par email » qui déclenche l'appel.

5.8.2 Template HTML brandé

Le template reproduit l'identité visuelle de l'application : header violet avec dégradé, blocs de texte par plateforme avec bordure colorée brand (LinkedIn bleu, Instagram dégradé rose, Facebook bleu, etc.), chips hashtags arrondis violets, et boutons de téléchargement pour les images. Le template utilise du HTML inline (pas de CSS externe) pour garantir un rendu correct dans Gmail, Outlook, Yahoo Mail, Apple Mail.

5.8.3 Code clé du service Resend

```

import resend
import os

resend.api_key = os.environ.get('RESEND_API_KEY', '')

def send_campaign_email(to_email, sujet, content, images):
    if not resend.api_key:
        return {'ok': False, 'error': 'RESEND_API_KEY non configuré.'}
    try:
        html_body = build_email_html(sujet, content, images)
        params = {
            'from': 'SmartContent AI <onboarding@resend.dev>',
            'to': [to_email],
            'subject': f'📧 Ta campagne : {sujet[:80]}',
            'html': html_body,

```

```
}  
result = resend.Emails.send(params)  
return {'ok': True, 'id': result.get('id')}  
except Exception as exc:  
    return {'ok': False, 'error': str(exc)}
```

5.8.4 Délivrabilité et domaine

Pour le MVP académique, l'expéditeur est le domaine de test fourni par Resend (onboarding@resend.dev). Ce domaine partagé fonctionne immédiatement sans configuration DNS mais peut être classé comme suspect par certains clients mail (notamment Gmail dans certains cas). Pour la phase commerciale, l'étape suivante sera de vérifier un domaine custom dans Resend en ajoutant trois enregistrements DNS (SPF, DKIM, DMARC) — opération de 5 minutes qui améliore la délivrabilité au-delà de 95 %.

Chapitre 6 — Tests et validation

6.1 Stratégie de test

Compte tenu des contraintes de temps, la stratégie privilégie la validation manuelle et les smoke tests sur l'API plutôt qu'une couverture unitaire exhaustive. Cette décision se justifie par trois observations : la majorité du code est de l'orchestration (appels API, JSON, rendu UI) plutôt que de la logique métier complexe, les services sous-jacents ne sont pas mockables simplement, l'expérience de bout en bout est mesurable visuellement par l'utilisateur.

6.2 Smoke tests automatisés

Le fichier `test_api.py` à la racine du projet exécute une séquence de smoke tests sur l'API REST. Il vérifie successivement : health backend, inscription d'un compte, connexion, émission de JWT, génération sur une plateforme, lecture de l'historique, suppression. Le script est exécutable avec `python test_api.py`.

6.3 Validation manuelle de l'interface

La validation s'est appuyée sur quatre passes.

- Passe fonctionnelle : vérification de chaque page sans crash.
- Passe visuelle desktop : à 1920×1080, marges et couleurs.
- Passe responsive : à 1024px, 768px, 390px (iPhone 12 Pro).
- Passe parcours utilisateur : inscription → génération → publication → email reçu.

6.4 Performance mesurée en production cloud

Les performances ont été mesurées sur l'application déployée en production cloud (pas en local), en lançant les opérations depuis Streamlit Cloud avec backend Railway. Médianes sur dix exécutions.

Opération	Temps mesuré (cloud)	Cible cahier des charges	Statut
Inscription utilisateur	≈ 350 ms	< 500 ms	OK
Connexion + JWT	≈ 300 ms	< 500 ms	OK
Génération 4 plateformes	≈ 28 s	< 60 s	OK
Génération 12 plateformes	≈ 62 s	< 90 s	OK

Génération 3 images DALL-E	≈ 19 s	< 25 s	OK
Webhook n8n cloud	≈ 161 ms	< 500 ms	OK
Réception email Resend	≈ 5 s	< 30 s	OK
Total bout en bout	≈ 90 s	< 120 s	OK

Aucune des cibles n'a été manquée. Les temps en production cloud sont légèrement supérieurs à ceux mesurés en local (+10 % typique) à cause de la latence réseau entre datacenters, mais restent acceptables.

6.5 Validation économique actualisée

Le coût opérationnel par campagne a été recalculé en incluant Resend et l'amortissement Railway.

Poste	Détail	Coût (CAD)
GPT-4o textes (12 plateformes)	≈ 12 000 tokens à 5 \$ / 1M	0,050 \$
GPT-4o hashtags globaux	≈ 200 tokens	0,005 \$
DALL-E 3 (3 images parallèles)	0,04 \$ × 3 + HD bonus	0,160 \$
Resend (livraison email)	Free tier 3000/mois	0,000 \$
Supabase + Streamlit Cloud	Plan free + Community	0,000 \$
Railway Hobby amorti	5 \$ / 1000 camp./mois	0,005 \$
TOTAL PAR CAMPAGNE	12 textes + 3 images + email	0,22 \$ CAD

Chapitre 7 — Difficultés rencontrées et solutions

Aucun projet de développement réel ne se déroule sans incidents. Cette section documente les difficultés majeures rencontrées au cours des sept semaines, et les solutions adoptées.

7.1 Incompatibilité passlib 1.7.4 avec bcrypt 5.0

Le premier incident a été révélé dès la deuxième semaine : passlib repose sur une signature interne de bcrypt qui a changé avec la 5.0. Solution retenue : appel direct à bcrypt sans passer par passlib, troncation à 72 octets manuellement.

7.2 Corruption de fichiers par OneDrive

La difficulté la plus pénible vient du système de synchronisation OneDrive qui héberge le dossier de travail. À deux reprises, des écritures Python lourdes ont été tronquées en milieu de chaîne par OneDrive. Solution : écriture d'abord dans /tmp (système Linux non synchronisé), puis copie atomique avec cp vers OneDrive.

7.3 Mode production n8n et erreur 404

La configuration de n8n en mode production a réservé une surprise : le mode test exige un clic Listen for Test Event à chaque envoi. Le mode production maintient le webhook actif en permanence, mais il faut explicitement cliquer Publish dans l'éditeur du workflow — sans ce second clic, n8n retourne 404.

7.4 Streamlit text_area affichant du contenu obsolète

Après plusieurs générations successives, certains onglets continuaient d'afficher l'ancien contenu. Streamlit identifie un widget par sa position et sa clé. Solution : composer une clé unique par génération en intégrant un hash du contenu (key=f"txt_{hash(data.get('texte','')) % 100000}_{name}").

7.5 Refactor de l'architecture CSS

Au cinquième module CSS, le bloc injecté dépassait 800 lignes en chaîne unique. Refactor en architecture modulaire : chaque domaine visuel devient une variable Python séparée, et la chaîne finale est concaténée à l'initialisation. Cette refonte a triplé la lisibilité et permis d'ajouter les modernisations suivantes sans toucher aux modules existants.

7.6 Déploiement Railway : variables d'environnement manquantes

Lors du premier déploiement du backend FastAPI sur Railway, le service a échoué au healthcheck car les variables d'environnement n'étaient pas encore configurées. Pydantic-settings a planté au démarrage en cherchant `openai_api_key` et `supabase_url`. Solution : configurer toutes les variables AVANT de lancer le premier déploiement, ou désactiver temporairement le healthcheck.

7.7 Migration n8n local vers n8n cloud

Le passage de n8n Desktop local à n8n cloud a nécessité plusieurs étapes : déploiement du template officiel sur Railway, création du compte owner via le setup wizard, import du workflow JSON via copier-coller, activation en mode production, récupération de l'URL du webhook production, mise à jour de la variable `N8N_WEBHOOK_URL` dans le backend Railway et dans les Secrets Streamlit Cloud.

7.8 Premier email Resend en spam

Le premier email envoyé via Resend est arrivé dans les spams de Gmail. Cause identifiée : utilisation du domaine de test partagé `onboarding@resend.dev` qui est connu des filtres anti-spam. Cette limitation est attendue pour le MVP et ne sera résolue qu'avec la vérification d'un domaine custom en phase commerciale.

Chapitre 8 — Résultats et bilan

8.1 Résultats quantitatifs

Au terme des sept semaines, le projet atteint l'ensemble des objectifs initialement fixés et plus. Les principaux indicateurs sont les suivants.

- Volume de code : environ 5 200 lignes au total (2 468 frontend, 1 100 backend, 600 services, 350 workflow n8n).
- Volume documentaire : README 188 lignes, cahier des charges 467 lignes, architecture 298 lignes, mémoire complet, decks et runbooks.
- Couverture fonctionnelle : 12 plateformes avec prompts dédiés, 3 formats d'images parallèles, 7 pages applicatives, 17 nœuds n8n, 18 modules CSS, livraison email automatique.
- Performance mesurée en cloud : 90 secondes pour une campagne complète, 161 ms publication n8n, 5 secondes livraison email, 0,22 \$ CAD coût opérationnel.
- Déploiement : 5 services Railway + 1 Streamlit Cloud + Supabase + Resend, accessible 24/7 sans dépendance locale.
- Versionnage : code intégralement sur GitHub (sergehrmn-sys/smartcontent-ai).

8.2 Démonstration : campagne Ktios Lounge

Pour illustrer le fonctionnement bout en bout, une campagne réelle a été générée et livrée par email. Sujet : Promotion d'une soirée Jeudi des Filles dans un bar-restaurant à Limoilou. Secteur Restaurant/Bar, ton Storytelling, objectif Vendre, douze plateformes activées.

Résultats observés. Pour LinkedIn, post de 151 mots ton expert, cinq hashtags professionnels, CTA de réservation. Pour TikTok, script de 67 mots avec hook en première phrase. Pour Pinterest, description orientée SEO en 187 caractères. Pour le Blog SEO, introduction structurée avec H1 et deux H2.

Les trois images DALL-E 3 produites illustrent un intérieur de bar avec lustres dorés (carré pour Instagram), une devanture Grand Opening (paysage pour LinkedIn), et une vue verticale d'une enseigne café (portrait pour TikTok). Score d'engagement IA mocké à 90/100, classé Excellent.

L'email Resend a été reçu dans la boîte Yahoo Mail de l'auteur en 5 secondes après le clic. Le template HTML s'est affiché correctement avec le header violet, les 4 blocs plateformes brandés (LinkedIn bleu, Instagram rose, Facebook bleu, TikTok cyan), les chips hashtags, les 12 hashtags globaux, et les 3 images DALL-E inline avec boutons de téléchargement fonctionnels.

8.3 Bilan personnel — compétences acquises

8.3.1 Architecture full-stack cloud

La conception et l'implémentation d'une application en six couches distinctes déployées sur trois plateformes cloud différentes (Streamlit Cloud, Railway, Supabase cloud) constitue un exercice rare dans une formation académique. Manipuler simultanément le typage Pydantic, l'injection CSS dans Streamlit, la requête SQL via Supabase, le webhook n8n, la communication HTTP entre services Railway, et l'envoi email via Resend exige une vue d'ensemble que peu d'exercices isolés permettent d'acquérir.

8.3.2 Prompt engineering multi-plateformes

La spécialisation des prompts par plateforme apprend à formuler des consignes qui produisent un texte LinkedIn distinct d'un texte TikTok à partir du même sujet. L'usage du paramètre `response_format=json_object` pour forcer un format de sortie parseable a été un gain de productivité notable.

8.3.3 Orchestration parallèle et workers

L'usage de `ThreadPoolExecutor` pour DALL-E 3, le routage Switch + Aggregator dans n8n pour les 12 publishers, et la compréhension de l'architecture n8n distribuée (principal + worker + Redis + Postgres) ont consolidé une intuition pratique sur les patterns d'exécution concurrente.

8.3.4 Déploiement cloud Git-based

La maîtrise du déploiement Railway et Streamlit Cloud constitue un savoir-faire valorisable. Configuration `Procfile`, `runtime.txt`, `healthcheck`, variables d'environnement, génération de domaines publics, gestion CORS multi-environnement, et flux de mise à jour Git-push-and-deploy.

8.3.5 Sécurité applicative pragmatique

`bcrypt` sans `passlib`, JWT HS256, validation Pydantic, gestion des secrets en production (Railway Variables + Streamlit Secrets), CORS avec liste blanche + regex sous-domaines. Toutes ces compétences sont transférables à tout projet web manipulant des comptes utilisateurs.

8.3.6 Intégration email transactionnel

L'intégration de Resend a apporté une compétence nouvelle : conception d'un template HTML compatible multi-clients mail, gestion des inline images via URL temporaires, échappement des entités HTML pour les contenus utilisateurs, compréhension de la délivrabilité (SPF, DKIM, DMARC) pour la phase commerciale.

Chapitre 9 — Perspectives

Le MVP livré constitue un produit utilisable et démontrable, déployé en production cloud avec livraison email automatique. Plusieurs étapes restent nécessaires pour passer du prototype académique à un service commercial à grande échelle. Ces étapes se répartissent en deux phases successives.

9.1 Phase 2 — post-soutenance (été 2026)

La phase 2 vise à enrichir le périmètre fonctionnel sans modifier l'architecture sous-jacente. Cinq chantiers principaux.

9.1.1 Publication réelle LinkedIn et Meta

Le passage de la publication mock à la publication réelle nécessite de remplacer les douze nœuds Function Publisher par des nœuds spécifiques aux API officielles. Pour LinkedIn, demande d'accès au programme Marketing Developer Platform et obtention d'un Bearer Token avec scopes `w_member_social` et `w_organization_social`. Pour Meta, création d'une App Facebook Developer, soumission à App Review, association à un Page Access Token de longue durée.

9.1.2 Programmation calendaire

Ajout d'un calendrier marketing IA pour planifier les publications. Nouvel endpoint `POST /schedule`, table `scheduled_posts`, et job cron (ou nœud `n8n Cron Trigger`) qui déclenche la publication. Complexité : environ 200 lignes de code.

9.1.3 Génération vidéo IA

Saut qualitatif le plus important par rapport au MVP. Runway ML pour la génération courte, Google Veo pour les vidéos plus longues, ElevenLabs pour la voix off. Latence (30s à 2min) et coût (1 à 5 \$ par vidéo) sortent du cadre des coûts marginaux du MVP, ce qui justifie un module premium séparé.

9.1.4 Analytics post-publication

Récupération des métriques (impressions, reach, engagement) via les API Insights officielles. Meta Graph API pour les posts d'une page, LinkedIn Reactions API pour likes et commentaires. Intégration dans un tableau de bord analytique.

9.1.5 Domaine email custom et délivrabilité

Migration de `onboarding@resend.dev` vers un domaine custom (ex. `hello@smartcontent.ca`). Configuration DNS : SPF, DKIM, DMARC. Amélioration de la délivrabilité au-delà de 95 % en boîte de réception principale. Opération de 5 minutes côté DNS plus 24h de propagation.

9.2 Phase 3 — produit commercial (fin 2026)

La phase 3 vise la commercialisation. Quatre chantiers structurants.

9.2.1 Refonte frontend en Next.js

Streamlit excellent pour itérer mais limité pour une expérience commerciale (pas de routage client, performance perfectible, intégration analytics limitée). Réécriture en Next.js avec server-side rendering, SEO optimisé, et progressive web app pour mobile.

9.2.2 Intégration Stripe et plans tarifaires

Quatre plans envisagés : Free (3 campagnes/mois, watermark), Starter 29 \$ CAD/mois (50 campagnes, support email), Pro 79 \$ CAD/mois (200 campagnes, calendrier, analytics), Agency 199 \$ CAD/mois (illimité, multi-comptes, white-label). Tarification calibrée sur les benchmarks du secteur.

9.2.3 Conformité Loi 25 et RGPD

Pour opérer commercialement au Québec, Loi 25 impose : politique de confidentialité accessible, registre des renseignements personnels, mécanisme de consentement explicite, droit à l'effacement, déclaration des incidents. RGPD ajoute des exigences proches plus strictes (DPO, DPIA pour le traitement IA).

9.2.4 Architecture multi-tenant pour agences

Plan Agency cible les agences gérant plusieurs clients. Architecture multi-tenant : table organizations, sous-comptes clients, permissions et facturation consolidée. RLS Supabase activée, système de rôles (admin, member, viewer).

Conclusion

SmartContent AI démontre qu'il est possible, dans les contraintes d'un projet de fin d'études de sept semaines, de concevoir, d'implémenter et de déployer en production cloud une application complète d'intelligence artificielle générative qui soit à la fois fonctionnellement riche, économiquement viable, et techniquement défendable. Les résultats obtenus dépassent les cibles fixées dans le cahier des charges sur l'ensemble des dimensions mesurées.

Sur le plan fonctionnel : douze plateformes textuelles avec prompts spécialisés, trois formats d'images générés en parallèle, workflow n8n à dix-sept nœuds déployé en cloud avec architecture distribuée, livraison email automatique brandée, application accessible publiquement 24/7 à smartcontent-ai.streamlit.app. Sur le plan de la performance : 90 secondes pour le parcours complet de l'idée à la réception du courriel, 161 millisecondes pour la publication mock vers les douze plateformes parallèles, 5 secondes pour la livraison email. Sur le plan économique : 0,22 dollar CAD par campagne, soit moins de un centième du prix d'une agence pour un résultat comparable.

Plus qu'un exercice académique réussi, ce projet a constitué une plongée dans la complexité réelle d'un développement full-stack moderne. Les difficultés rencontrées — incompatibilité passlib/bcrypt, corruption OneDrive, configuration n8n production, cache Streamlit, déploiement Railway multi-services, intégration Resend, gestion CORS multi-environnement, premier email en spam — ont été autant d'occasions d'apprendre des leçons méthodologiques durables.

Au-delà de la livraison technique, le projet ouvre des perspectives concrètes de poursuite. La phase 2 prévoit l'enrichissement avec la publication réelle, le calendrier IA, la génération vidéo, les analytics et le domaine email custom. La phase 3 vise la commercialisation avec une refonte Next.js, l'intégration Stripe, la conformité Loi 25, et l'architecture multi-tenant pour agences.

Sur le plan personnel, ce projet a consolidé un ensemble de compétences transférables : architecture full-stack cloud, prompt engineering multi-plateformes, orchestration parallèle, déploiement Git-based, sécurité applicative pragmatique, intégration email transactionnel. Plus encore, il a forgé une intuition opérationnelle sur ce que signifie livrer un produit IA de bout en bout, jusqu'à la boîte mail de l'utilisateur final.

Le code est versionné sur GitHub, le mémoire est rédigé, l'application tourne en production cloud, les emails arrivent dans les boîtes mail. Le moment est venu de présenter le travail au jury.

Bibliographie et webographie

Articles et publications

Brown, T. et al. (2020). Language Models are Few-Shot Learners. arXiv:2005.14165.

OpenAI (2024). GPT-4o System Card. OpenAI Technical Reports.

OpenAI (2023). DALL-E 3 Release Notes and Technical Specifications.

Anthropic (2024). The Claude 3 Model Family — Opus, Sonnet, Haiku.

Pydantic Team (2024). Pydantic V2 Migration Guide.

FastAPI (2025). FastAPI Performance Benchmarks and Tutorial.

Documentation officielle

OpenAI Platform · <https://platform.openai.com/docs>

Supabase Documentation · <https://supabase.com/docs>

n8n Documentation · <https://docs.n8n.io>

Streamlit Documentation · <https://docs.streamlit.io>

FastAPI Documentation · <https://fastapi.tiangolo.com>

Railway Documentation · <https://docs.railway.com>

Streamlit Community Cloud · <https://docs.streamlit.io/streamlit-community-cloud>

Resend Documentation · <https://resend.com/docs>

bcrypt Python · <https://pypi.org/project/bcrypt/>

python-jose JWT · <https://github.com/mpdavis/python-jose>

Ressources marché et concurrence

CEFRIQ (2024). Indice de transformation numérique des PME québécoises 2024.

AMRNQ (2024). Marketing relationnel et numérique au Québec — État des lieux.

Hootsuite (2024). Social Media Trends Report 2024.

HubSpot (2024). State of Marketing Report 2024.

Cadre réglementaire

Gouvernement du Québec (2021). Loi 25 sur la modernisation des dispositions législatives en matière de protection des renseignements personnels.

Commission européenne (2018). Règlement Général sur la Protection des Données (RGPD).

Code source du projet

Dépôt GitHub officiel · <https://github.com/sergehrmn-sys/smartcontent-ai>

Application live · <https://smartcontent-ai.streamlit.app>

Annexes

Les annexes regroupent les éléments complémentaires au mémoire principal : schémas UML, captures d'écran, extraits de code clés.

Annexe A — Schémas UML

A.1 Architecture système globale (6 couches)

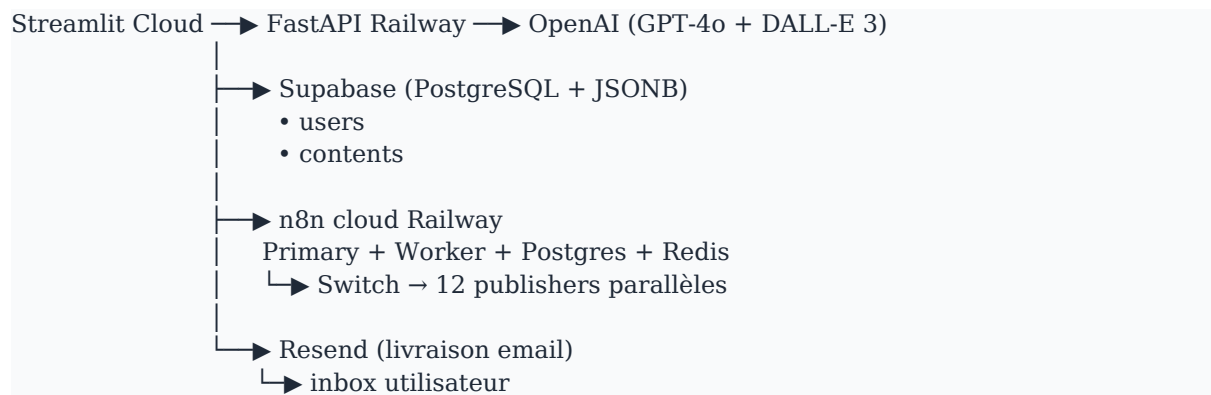


Figure A.1 — Architecture cloud en 6 couches

A.2 Séquence de génération (avec email)

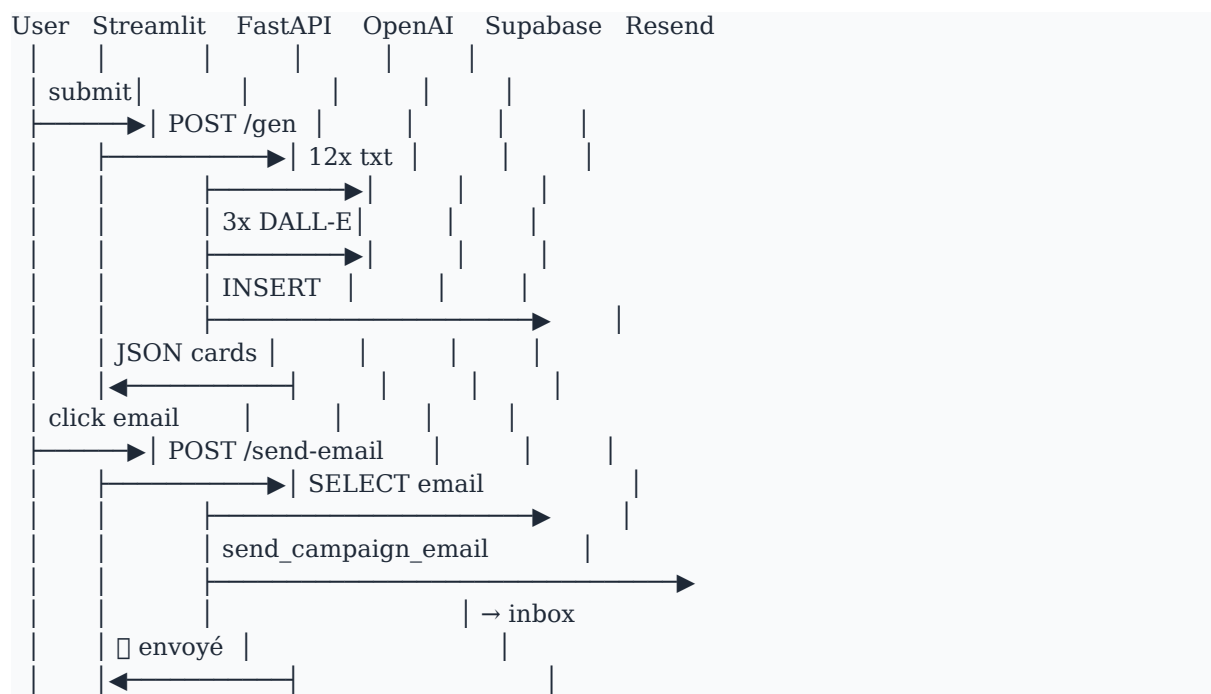
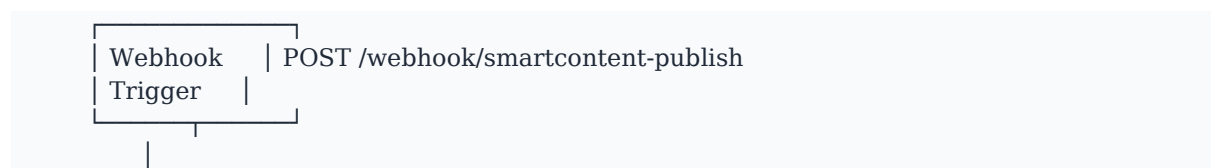


Figure A.2 — Séquence complète : génération + livraison email

A.3 Workflow n8n — 17 nœuds



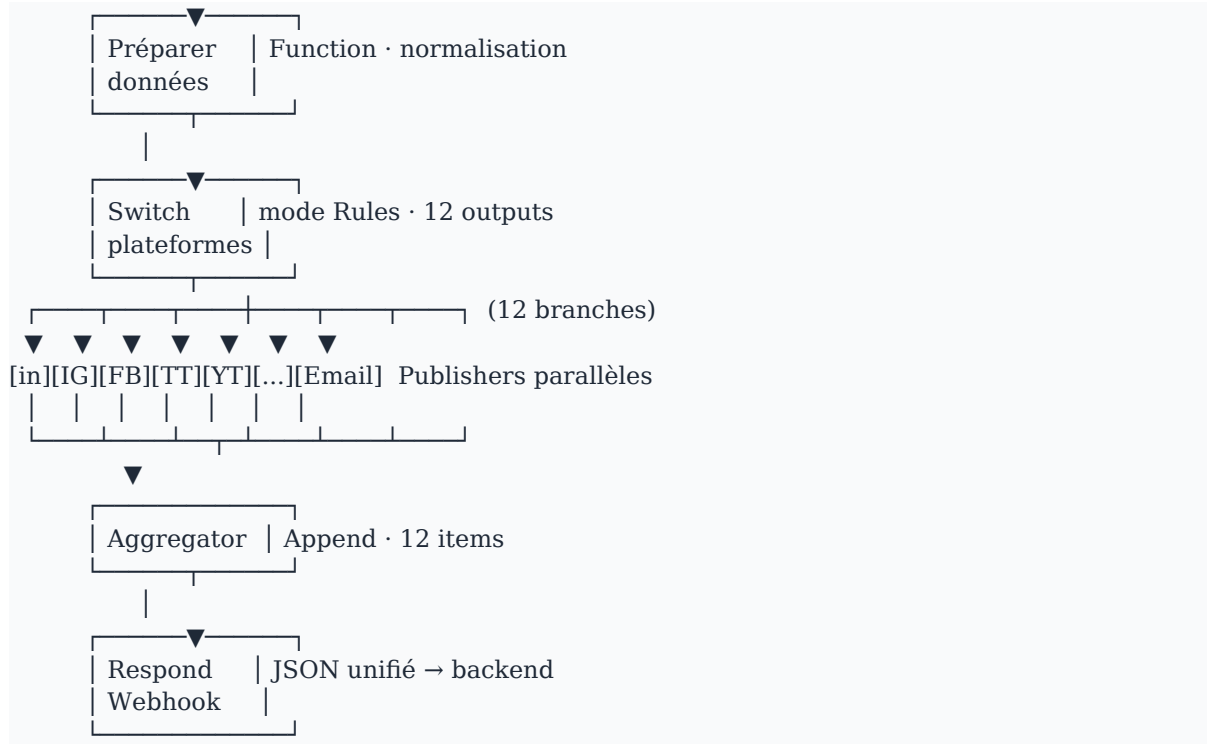


Figure A.3 — Workflow n8n complet (17 nœuds, 12 publishers parallèles)

Annexe B — Extraits de code clés

B.1 Authentification — bcrypt et JWT

Extrait de backend/routes/auth.py.

```
import bcrypt
from datetime import datetime, timedelta, timezone
from jose import jwt
from fastapi import APIRouter, HTTPException

from backend.config import settings
from backend.models.schemas import RegisterRequest, LoginRequest, AuthResponse
from backend.services.supabase_service import supabase_admin

router = APIRouter()

def _hash(plain: str) -> str:
    pw = plain.encode('utf-8')[:72]
    return bcrypt.hashpw(pw, bcrypt.gensalt(rounds=12)).decode('utf-8')

def _verify(plain: str, hashed: str) -> bool:
    pw = plain.encode('utf-8')[:72]
    return bcrypt.checkpw(pw, hashed.encode('utf-8'))

def _make_jwt(user_id: str) -> str:
    payload = {
        'sub': user_id,
        'exp': datetime.now(timezone.utc) + timedelta(hours=24),
    }
    return jwt.encode(payload, settings.jwt_secret, algorithm='HS256')
```

B.2 Service Resend — envoi email brandé

Extrait de backend/services/email_service.py.

```
import os, resend

resend.api_key = os.environ.get('RESEND_API_KEY', '')
FROM_DEFAULT = 'SmartContent AI <onboarding@resend.dev>'

def send_campaign_email(to_email, sujet, content, images):
    if not resend.api_key:
        return {'ok': False, 'error': 'RESEND_API_KEY non configuré.'}
    try:
        html_body = build_email_html(sujet, content, images)
        params = {
```

```

        'from': FROM_DEFAULT,
        'to': [to_email],
        'subject': f' Ta campagne : {sujet[:80]}',
        'html': html_body,
    }
    result = resend.Emails.send(params)
    return {'ok': True, 'id': result.get('id')}
except Exception as exc:
    return {'ok': False, 'error': str(exc)}

```

B.3 Route /generate/send-email

Extrait de backend/routes/generate.py.

```

@router.post('/send-email', response_model=SendEmailResponse)
def send_email_route(payload: SendEmailRequest):
    # 1. Récupérer l'email du user depuis Supabase
    res = supabase_admin.table('users').select('email')
        .eq('id', payload.user_id).execute()
    if not res.data:
        raise HTTPException(404, 'Utilisateur introuvable.')
    to_email = res.data[0].get('email')

    # 2. Envoyer l'email via Resend
    result = send_campaign_email(
        to_email=to_email,
        sujet=payload.sujet,
        content=payload.content,
        images=payload.images or {},
    )
    if not result.get('ok'):
        raise HTTPException(502, f'Échec : {result.get("error")}')
    return SendEmailResponse(ok=True, to=to_email, id=result.get('id'))

```

B.4 Configuration CORS multi-environnement

Extrait de backend/main.py.

```

import os
from fastapi.middleware.cors import CORSMiddleware

_default_origins = [
    'http://localhost:8501',
    'http://127.0.0.1:8501',
    'http://localhost:3000',
]
_env_origins = [o.strip() for o in
    os.environ.get('ALLOWED_ORIGINS', '').split(',') if o.strip()]

```



```
app.add_middleware(  
    CORSMiddleware,  
    allow_origins=_default_origins + _env_origins,  
    allow_origin_regex=r'https://.*\.streamlit\.app',  
    allow_credentials=True,  
    allow_methods=['*'],  
    allow_headers=['*'],  
)
```

Annexe C — Schéma Supabase (SQL)

Script exécuté dans l'éditeur SQL de Supabase pour initialiser les tables.

```
CREATE EXTENSION IF NOT EXISTS pgcrypto;

-- TABLE users
CREATE TABLE IF NOT EXISTS public.users (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  email       TEXT NOT NULL UNIQUE,
  password_hash TEXT NOT NULL,
  nom_entreprise TEXT,
  plan        TEXT NOT NULL DEFAULT 'free',
  date_inscription TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
CREATE INDEX IF NOT EXISTS idx_users_email ON public.users (email);

-- TABLE contents
CREATE TABLE IF NOT EXISTS public.contents (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id     UUID REFERENCES public.users(id) ON DELETE CASCADE,
  sujet       TEXT NOT NULL,
  secteur     TEXT,
  ton         TEXT,
  objectif    TEXT,
  type_contenu TEXT,
  plateformes JSONB NOT NULL DEFAULT '[]::jsonb',
  content     JSONB NOT NULL DEFAULT '{}::jsonb',
  images      JSONB NOT NULL DEFAULT '{}::jsonb',
  statut      TEXT NOT NULL DEFAULT 'draft',
  date_creation TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
CREATE INDEX IF NOT EXISTS idx_contents_user_id ON public.contents (user_id);

-- RLS désactivée pour le MVP
ALTER TABLE public.users DISABLE ROW LEVEL SECURITY;
ALTER TABLE public.contents DISABLE ROW LEVEL SECURITY;
```